



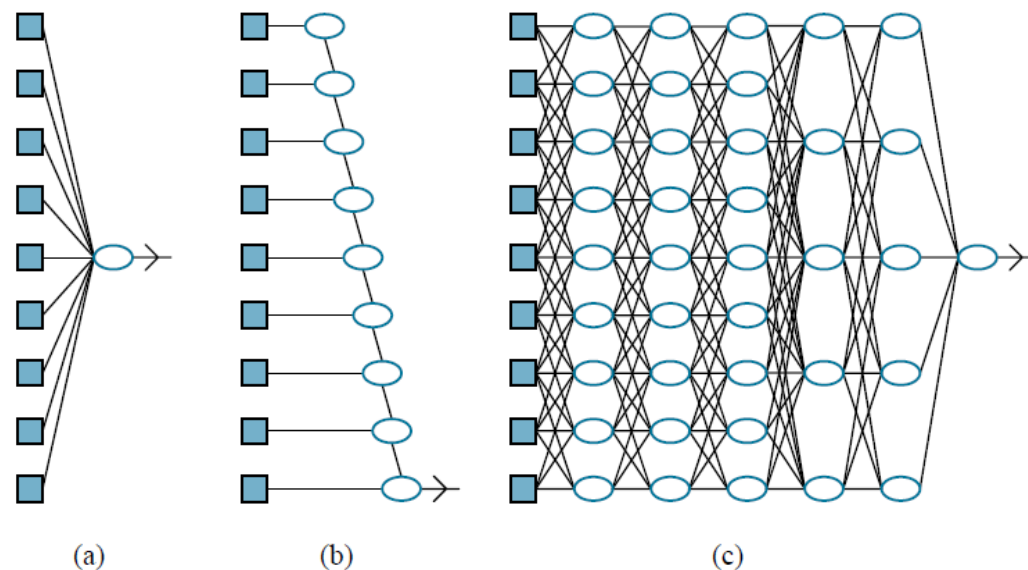
UMĚLÁ INTELIGENCE

Přednáška 8

Hluboké učení, neuronové sítě

HLUBOKÉ UČENÍ - DEEP LEARNING (DL)

- DL = skupina technik pro strojové učení, kde hypotézy mají formu mnohavrstvých algebraických obvodů s nastavitelnou silou propojení jednotlivých prvků sítě
 - model - neuronová síť (NN) - reprezentuje složenou nelineární funkci s velkým počtem parametrů
 - NN s více vrstvami jsou schopné vytvářet postupně bohatší reprezentace vstupních dat
 - zpracování nestrukturovaných dat: obraz, řeč, přirozený jazyk, video nebo velmi rozsáhlé signály
 - typické aplikace:
 - Počítačové vidění: detekce objektů, segmentace obrazu, rozpoznávání rukopisu.
 - Zpracování řeči: automatický přepis, rozpoznávání mluvčího, syntéza hlasu.
 - Zpracování textu: strojový překlad, klasifikace dokumentů, sumarizace, chatboti.
 - Průmysl a věda: predikce časových řad, analýza senzorických dat, diagnostika poruch, bioinformatika.
-



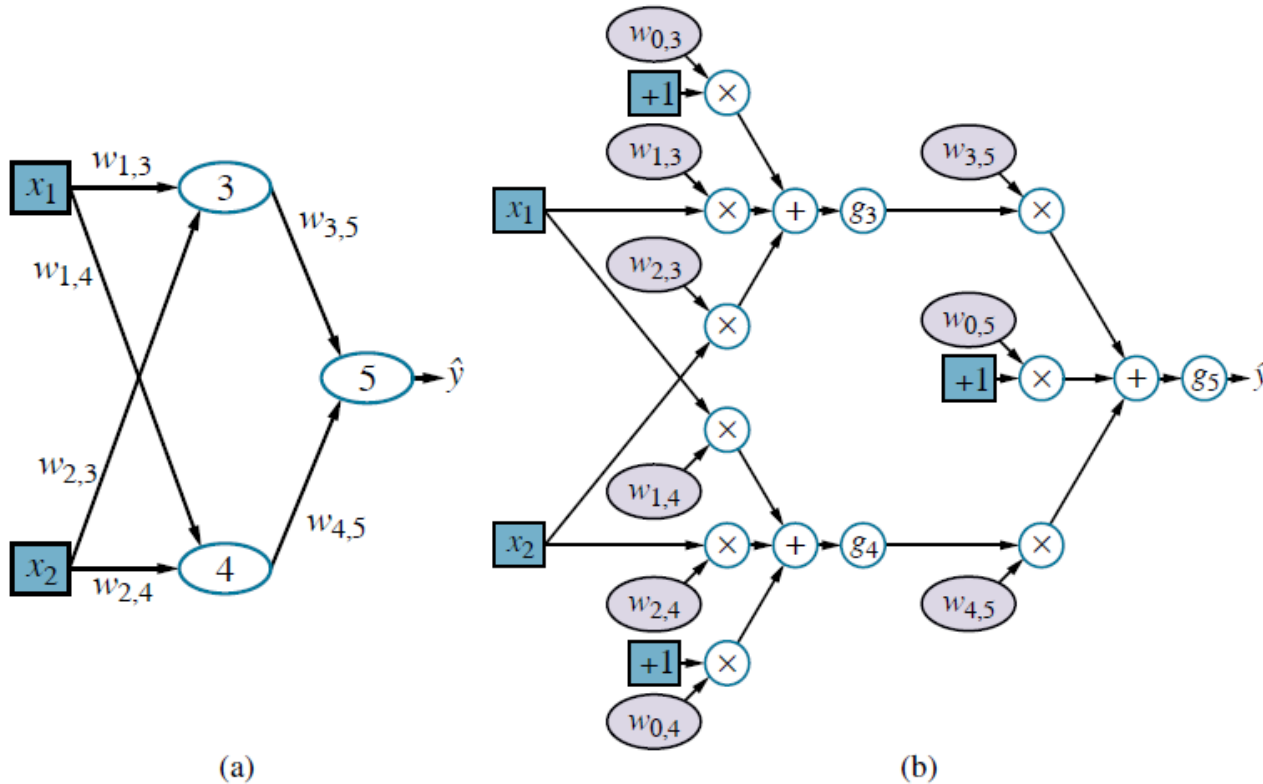
(a) lineární regrese
(b) rozhodovací strom
(c) neuronová síť (NS)

- lineární nebo logistická regrese umí zpracovat velký počet vstupních proměnných, ale výpočetní cesta od vstupů k výstupu je velmi krátká a vstupní proměnné jsou navzájem nezávislé --> omezená vyjadřovací schopnost --> tyto modely mohou reprezentovat pouze lineární funkce a hranice ve vstupním prostoru
- rozhodovací seznamy a rozhodovací stromy - umožňují nelineární větvení, ale jejich výpočetní cesta je lokálně hluboká jen pro malou část vstupního prostoru
- neuronová síť kombinuje dlouhou výpočetní cestu s tím, že informace z různých vstupů se mohou průběžně míchat a společně ovlivňovat výstup

Dopředná neuronová síť

Dopředná (feedforward) neuronová síť je orientovaný acyklický graf. Hodnoty postupují od vstupní vrstvy přes jednu či více skrytých vrstev k vrstvě výstupní. Každá jednotka ve skryté nebo výstupní vrstvě spočítá vážený součet svých vstupů a na něj aplikuje aktivační funkci. Uzly grafu se nazývají jednotky. Pomocný vstup $x_0 = 1$ reprezentuje bias, tedy posun rozhodovací hranice. Bez biasu by síť musela procházet počátkem a zbytečně by se omezila.

Dopředná neuronová síť



Příklad dopředné NN se dvěma vstupy, skrytou vrstvou a jedním výstupem

Dopředná (feedforward) neuronová síť je orientovaný acyklický graf. Hodnoty postupují od vstupní vrstvy přes jednu či více skrytých vrstev k vrstvě výstupní. Každá jednotka ve skryté nebo výstupní vrstvě spočítá vážený součet svých vstupů a na něj aplikuje aktivační funkci. Pomocný vstup $x_0 = 1$ reprezentuje bias, tedy posun rozhodovací hranice. Bez biasu by síť musela procházet počátkem a zbytečně by se omezila.

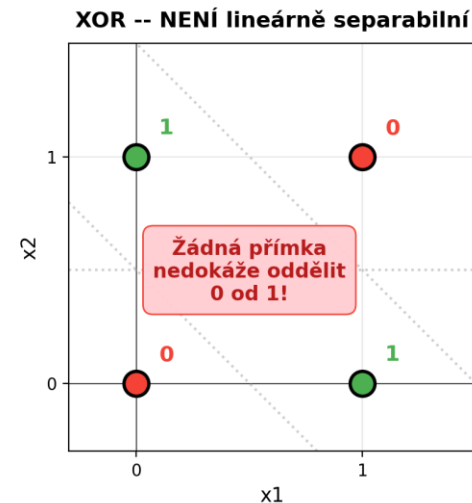
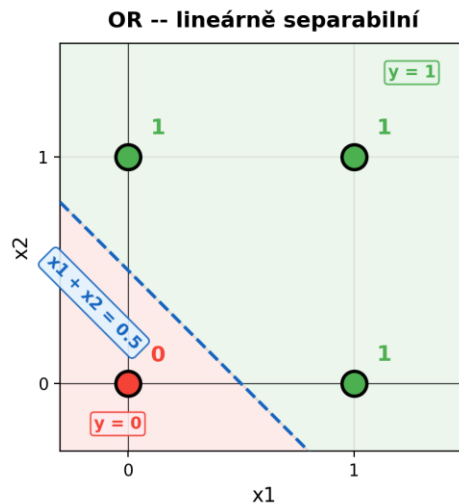
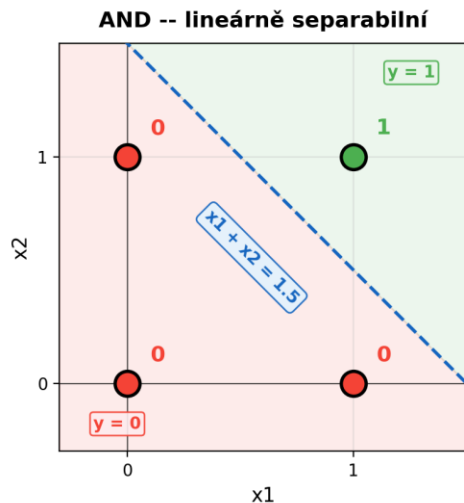
POZOR - bez nelineární aktivační funkce by se více lineárních vrstev složilo opět jen do jedné lineární transformace. Hloubka by pak nepřinášela žádnou dodatečnou vyjadřovací sílu.

Příklad: síť pro problém XOR

Problém XOR je klasická ukázka toho, že jedna lineární rozhodovací hranice nestačí. Body (0,0) a (1,1) patří do jedné třídy, zatímco (0,1) a (1,0) do druhé. Jednovrstvý perceptron takový problém nerozliší.

Jakmile však přidáme skrytou vrstvu se dvěma neurony, může jeden neuron rozpoznat oblast „alespoň jeden vstup je 1“ a druhý „oba vstupy nejsou současně 1“. Výstupní neuron pak tyto podmínky kombinuje. Tento příklad dobře ukazuje, proč skryté vrstvy zvyšují vyjadřovací schopnost modelu.

Lineární separabilita -- proč XOR potřebuje skrytou vrstvu



DOPŘEDNÉ NEURONOVÉ SÍTĚ

- jednosměrné propojení
- reprezentace pomocí orientovaného acyklického grafu s určenými vstupními a výstupními uzly
- uzly v síti se nazývají jednotky
- jednotka počítá vážený součet vstupů z předchozích uzlů, na tento součet se aplikuje nelineární **aktivační funkce** g_j

$$a_j = g_j(\sum_i w_{i,j} a_i) \equiv g_j(in_j)$$

kde $w_{i,j}$ je váha na spojnici od jednotky i k jednotce j

- každá jednotka má pomocný vstup $x_0 = 1$, a tedy vztah lze zapsat vektorově:

$$a_j = g_j(\mathbf{w}^T \mathbf{x})$$

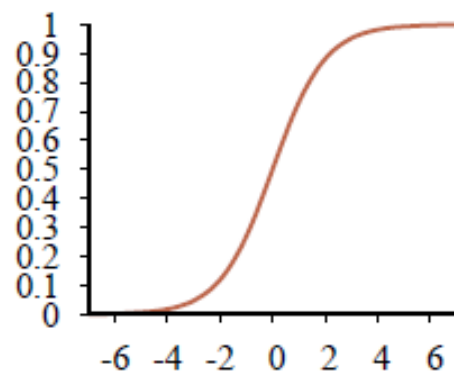
AKTIVAČNÍ FUNKCE

(a) sigmoidní (logistická) $\sigma(x) = 1/(1 + e^{-x})$

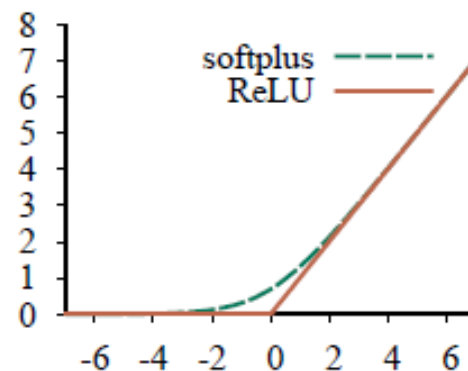
(b) ReLU – oříznutá lineární $\text{ReLU}(x) = \max(0, x)$

softplus – hladká verze ReLU $\text{softplus}(x) = \log(1 + e^x)$

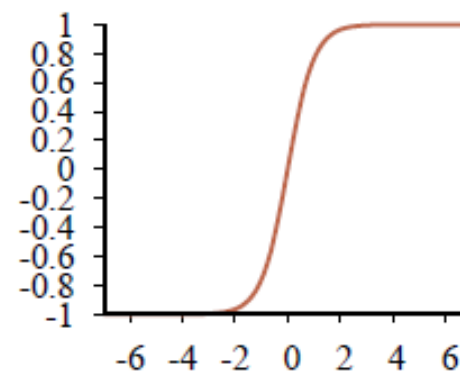
(c) hyperbolická tangenta - rozšíření oboru hodnot sigmoidy (a) na interval od -1 do +1



(a)



(b)



(c)

DOPŘEDNÁ NEURONOVÁ SÍŤ

Spojením více jednotek do sítě vzniká složená funkce, která se skládá z algebraických výrazů reprezentovaných jednotlivými jednotkami. Například síť znázorněná na předchozí straně představuje funkci $\mathbf{h}_{\mathbf{w}}(\mathbf{x})$ parametrizovanou váhami $\mathbf{w}$, která zobrazuje dvouprvkový vstupní vektor $\mathbf{x}$ na skalární výstupní hodnotu $\hat{y}$. Vnitřní struktura funkce odpovídá struktuře sítě.

$$\begin{aligned}\hat{y} &= g_5(in_5) = g_5(w_{0,5} + w_{3,5} a_3 + w_{4,5} a_4) & \hat{y} &= h_{\mathbf{w}}(\mathbf{x}) \\ &= g_5(w_{0,5} + w_{3,5} g_3(in_3) + w_{4,5} g_4(in_4)) \\ &= g_5(w_{0,5} + w_{3,5} g_3(w_{0,3} + w_{1,3} x_1 + w_{2,3} x_2) \\ &\quad + w_{4,5} g_4(w_{0,4} + w_{1,4} x_1 + w_{2,4} x_2))\end{aligned}$$

Vektorový zápis: $h_{\mathbf{w}}(\mathbf{x}) = \mathbf{g}^{(2)}(\mathbf{W}^{(2)} \mathbf{g}^{(1)}(\mathbf{W}^{(1)} \mathbf{x}))$

GRADIENTY A UČENÍ SÍTĚ

- metoda učení s učitelem založená sestupným gradientu, tj. výpočet gradientu ztrátové funkce vzhledem k vahám a úprava vah ve směru klesání gradientu tak, aby se ztráta snižovala, viz Přednášky č. 7, 4
- pro výpočet vah vedoucích do jednotek ve výstupní vrstvě je postup stejný jako v P7, pro váhy vedoucí do jednotek ve skrytých vrstvách, které nejsou přímo připojeny k výstupům, je víckrát aplikováno řetězové pravidlo
- kvadratická ztrátová funkce $Loss(h_{\mathbf{w}}) = L_2(y, h_{\mathbf{w}}(\mathbf{x})) = \|y - h_{\mathbf{w}}(\mathbf{x})\|^2 = (y - \hat{y})^2$
- řetězové pravidlo $\partial g(f(x)) / \partial x = g'(f(x)) \partial f(x) / \partial x$

GRADIENTY A UČENÍ SÍTĚ

- výpočet gradientu pro výstupní vrstvu:

$$\begin{aligned}\frac{\partial}{\partial w_{3,5}} \text{Loss}(h_{\mathbf{w}}) &= \frac{\partial}{\partial w_{3,5}} (y - \hat{y})^2 = -2(y - \hat{y}) \frac{\partial \hat{y}}{\partial w_{3,5}} \\ &= -2(y - \hat{y}) \frac{\partial}{\partial w_{3,5}} g_5(in_5) = -2(y - \hat{y}) g'_5(in_5) \frac{\partial}{\partial w_{3,5}} in_5 \\ &= -2(y - \hat{y}) g'_5(in_5) \frac{\partial}{\partial w_{3,5}} (w_{0,5} + w_{3,5} a_3 + w_{4,5} a_4) \\ &= -2(y - \hat{y}) g'_5(in_5) a_3 .\end{aligned}$$

GRADIENTY A UČENÍ SÍTĚ

- výpočet gradientu pro vnitřní vrstvu:

$$\begin{aligned}\frac{\partial}{\partial w_{1,3}} \text{Loss}(h_{\mathbf{w}}) &= -2(y - \hat{y}) g'_5(in_5) \frac{\partial}{\partial w_{1,3}} (w_{0,5} + w_{3,5} a_3 + w_{4,5} a_4) \\ &= -2(y - \hat{y}) g'_5(in_5) w_{3,5} \frac{\partial}{\partial w_{1,3}} a_3 \\ &= -2(y - \hat{y}) g'_5(in_5) w_{3,5} \frac{\partial}{\partial w_{1,3}} g_3(in_3) \\ &= -2(y - \hat{y}) g'_5(in_5) w_{3,5} g'_3(in_3) \frac{\partial}{\partial w_{1,3}} in_3 \\ &= -2(y - \hat{y}) g'_5(in_5) w_{3,5} g'_3(in_3) \frac{\partial}{\partial w_{1,3}} (w_{0,3} + w_{1,3} x_1 + w_{2,3} x_2) \\ &= -2(y - \hat{y}) g'_5(in_5) w_{3,5} g'_3(in_3) x_1 .\end{aligned}$$

SHRNUTÍ - Učení s učitelem: ztráta, gradientní sestup a zpětné šíření chyby (backpropagation)

Učení s učitelem znamená, že pro každý trénovací vstup x známe také požadovaný výstup y . Síť vytvoří predikci $\hat{y}$ a rozdíl mezi y a $\hat{y}$ se vyjádří ztrátovou funkcí. Cílem trénování je nalézt takové váhy, které ztrátu na trénovacích datech minimalizují a zároveň vedou k dobré generalizaci na nových datech.

Velikost kroku řídí rychlost učení α . Příliš velká hodnota α vede k divergenci, příliš malá k pomalému trénování.

$$L = \frac{1}{N} \sum_{n=1}^N \ell(y_n, \hat{y}_n), \quad w \leftarrow w - \alpha \nabla_w L$$

Proč gradient? Gradient ukazuje směr nejrychlejšího růstu ztráty. Pokud chceme ztrátu snižovat, musíme jít opačným směrem.

Odvození backpropagation krok za krokem:

Nejprve zavedme notaci pro jednu váhu $w_{\{j,k\}}$ vedoucí z neuronu j do výstupního neuronu k :

- výstup neuronu j označme a_j ,
- vstup do výstupního neuronu k označme $in_k = \sum_j w_{\{j,k\}} a_j$,
- výstup sítě je $\hat{y}_k = g_k(in_k)$,
- ztrátu pro jeden vzor zapisujeme $L = 1/2 \cdot (y_k - \hat{y}_k)^2$. Faktor $1/2$ je čistě technický; usnadní derivaci.

Nyní chceme spočítat, jak se ztráta změní při malé změně váhy $w_{\{j,k\}}$. Použijeme řetězové pravidlo:

$$\partial L / \partial w_{\{j,k\}} = (\partial L / \partial \hat{y}_k) \cdot (\partial \hat{y}_k / \partial in_k) \cdot (\partial in_k / \partial w_{\{j,k\}})$$

Jednotlivé části spočítáme zvlášť:

- $\partial L / \partial \hat{y}_k = \hat{y}_k - y_k$, protože $L = 1/2 (y_k - \hat{y}_k)^2$,
 - $\partial \hat{y}_k / \partial in_k = g'_k(in_k)$, tedy derivace aktivační funkce ve výstupním neuronu,
 - $\partial in_k / \partial w_{\{j,k\}} = a_j$, protože in_k je lineární kombinace vstupů.
-

$$\delta_k = \frac{\partial L}{\partial in_k} = (\hat{y}_k - y_k) g_k'(in_k), \quad \frac{\partial L}{\partial w_{j,k}} = a_j \delta_k$$

Definujeme si tzv. lokální chybu výstupního neuronu $\delta_k = \partial L / \partial in_k$. Pak gradient podle libovolné vstupní váhy do tohoto neuronu získáme jednoduchým součinem aktivace předchozího neuronu a_j a lokální chyby δ_k .

Proč je potřeba chybu „šířit zpět“?

Ve skryté vrstvě neuron nevidí cílovou hodnotu přímo. Jeho chyba se projeví teprve přes neurony v dalších vrstvách. Proto musíme opět použít řetězové pravidlo, tentokrát přes všechny neurony, do nichž neuron j vede.

$$\delta_j = \partial L / \partial in_j = g_j'(in_j) \cdot \sum_k w_{j,k} \cdot \delta_k$$

Interpretace:

- součet $\sum_k w_{j,k} \cdot \delta_k$ říká, jak silně neuron j ovlivňuje chybu ve vyšší vrstvě,
 - násobení derivací $g_j'(in_j)$ říká, jak citlivý je samotný neuron j na změnu svého vstupu,
 - jakmile známe δ_j , gradient podle libovolné váhy z předchozí vrstvy je opět $a_i \cdot \delta_j$.
-

Backpropagation jako algoritmus

- 1. Proved' dopředný průchod a ulož si aktivace a_i a vstupy in_j ve všech vrstvách.
- 2. Na výstupu spočítej lokální chybu δ pro poslední vrstvu.
- 3. Postupuj vrstvami zpět a pro každý neuron spočítej jeho δ podle chyb následující vrstvy.
- 4. Z gradientů $\partial L / \partial w_{\{i,j\}} = a_i \cdot \delta_j$ sestav aktualizaci vah.
- 5. Proces opakuj pro další dávku dat až do konvergence nebo dosažení zastavovacího kritéria.

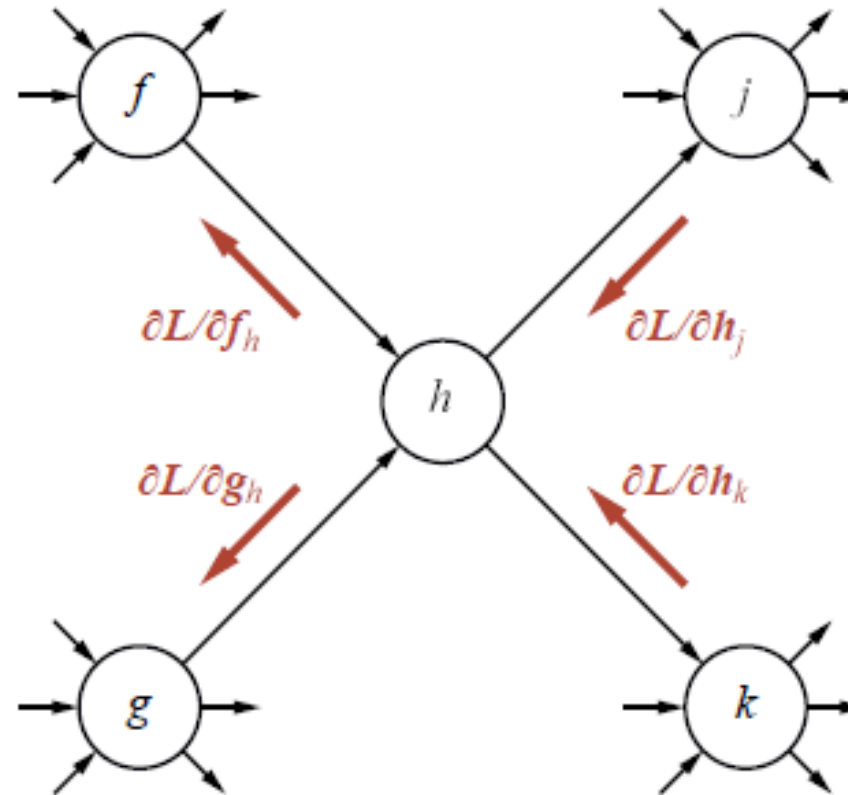
```
# Backpropagation pro MLP se dvěma vrstvami
for epoch in range(E):
    for (x, y) in training_batches:
        # 1) forward pass
        a0 = x
        in1 = W1 @ a0 + b1
        a1 = g1(in1)
        in2 = W2 @ a1 + b2
        y_hat = g2(in2)
        # 2) chyba ve výstupu
        delta2 = dLoss_dyhat(y, y_hat) * g2_prime(in2)
        # 3) chyba ve skryté vrstvě
        delta1 = (W2.T @ delta2) * g1_prime(in1)
        # 4) gradienty
        dW2 = outer(delta2, a1)
        db2 = delta2
        dW1 = outer(delta1, a0)
        db1 = delta1
        # 5) aktualizace
        W2 -= alpha * dW2
        b2 -= alpha * db2
        W1 -= alpha * dW1
        b1 -= alpha * db1
```

UČÍCÍ SE ALGORITMY

- metoda sestupného gradientu

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} L(\mathbf{w})$$

- metoda zpětného šíření chyby



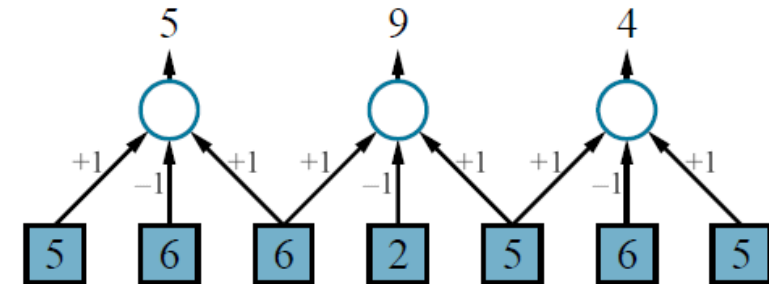
KONVOLUČNÍ NN - CONVOLUTIONAL NEURAL NETWORK (CNN)

- vhodná pro rozpoznávání obrazu – bere v úvahu závislost mezi sousedními pixely
- CNN je síť, která obsahuje (minimálně v prvních vrstvách) prostorově lokální spojení a má strukturu vah, která je kopírována mezi jednotkami v příslušné vrstvě; vzor vah, který je kopírován, se nazývá jádro **k** a proces aplikace jádra na pixely obrazu (nebo na prostorově uspořádané jednotky v následující vrstvě) se nazývá konvoluce.
- konvoluční vrstva – vstupy pouze od několika okolních uzlů

$$\mathbf{z} = \mathbf{x} * \mathbf{k}$$

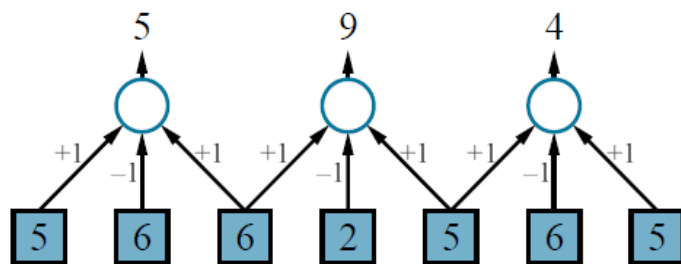
- výpočet:
$$z_i = \sum_{j=1}^l k_j x_{j+i-(l+1)/2}$$

- prostorová invariance
- jádro **k**, velikost jádra **l**, krok **s**



Příklad jednorozměrné konvoluční operace s jádrem o velikosti $l=3$ a krokem $s=2$.

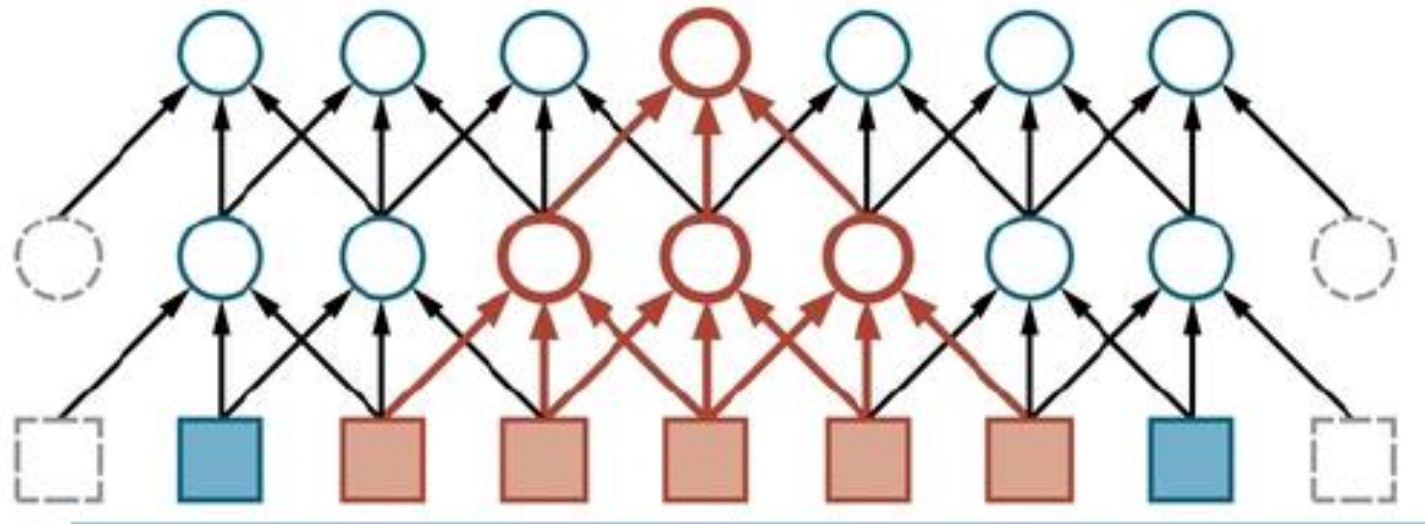
KONVOLUČNÍ NS



konvoluce jako maticové násobení

$$\overset{K}{\begin{pmatrix} +1 & -1 & +1 & 0 & 0 & 0 & 0 \\ 0 & 0 & +1 & -1 & +1 & 0 & 0 \\ 0 & 0 & 0 & 0 & +1 & -1 & +1 \end{pmatrix}} \begin{pmatrix} 5 \\ 6 \\ 6 \\ 2 \\ 5 \\ 6 \\ 5 \end{pmatrix} = \begin{pmatrix} 5 \\ 9 \\ 4 \end{pmatrix}$$

KONVOLUČNÍ NS



První dvě vrstvy konvoluční NS pro 1D obraz s velikostí jádra $l=3$ a krokem $s=1$.

Na levém a pravém okraji je přidán "padding", aby skryté vrstvy měly stejnou velikost jako vstup. Červeně je znázorněno zorné pole jednotky ve druhé skryté vrstvě. Obecně platí, že čím hlouběji je jednotka, tím větší je její zorné pole.

KONVOLUČNÍ NN - CONVOLUTIONAL NEURAL NETWORK (CNN)

Konvoluční neuronové sítě (CNN) jsou specializované architektury pro data s lokální strukturou. Typickým případem je obraz, kde sousední pixely nejsou nezávislé. CNN tuto skutečnost respektuje dvěma zásadními vlastnostmi: lokálními receptivními poli a sdílením vah.

Lokální receptivní pole znamená, že neuron v konvoluční vrstvě „vidí“ jen malou oblast vstupu. Sdílení vah znamená, že stejný filtr (jádro) se aplikuje na všech místech obrazu. To výrazně snižuje počet parametrů a zároveň zavádí prostorovou invarianci: stejný vzor lze detekovat kdekoli v obrazu.

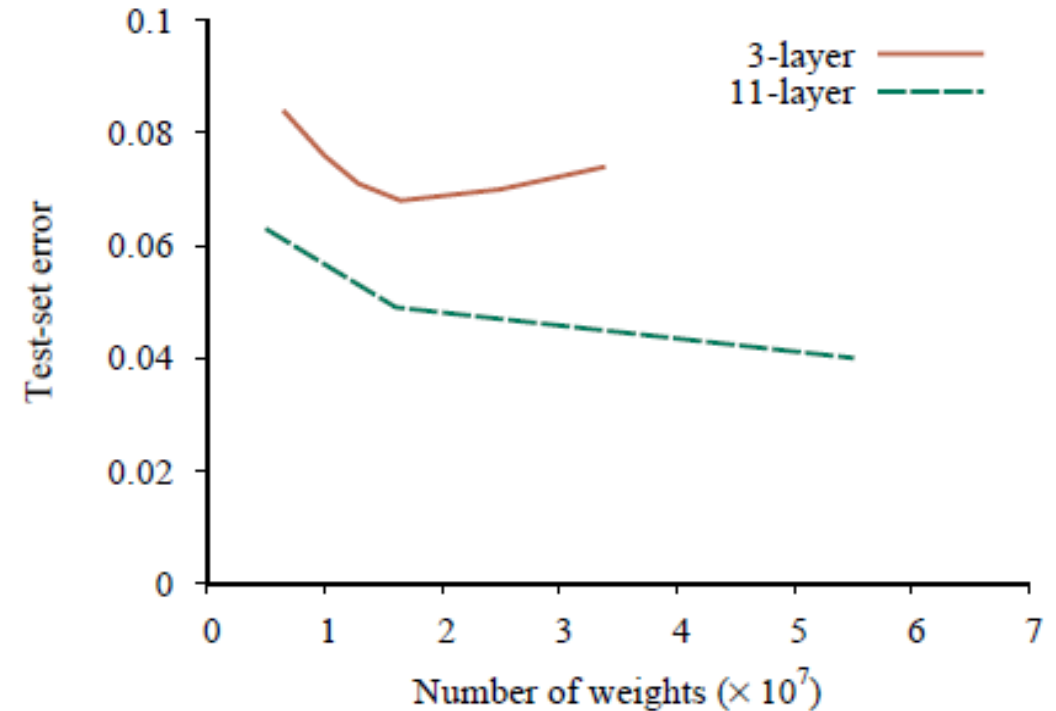
Pojem	Význam	Praktický dopad
Kernel / filtr	malá matice vah	detekuje lokální vzor (např. hranu, texturu)
Stride	o kolik se filtr posune	vyšší stride snižuje rozlišení a zrychluje výpočet
Padding	doplnění okrajů	umožní zachovat rozměr nebo lépe využít okrajové pixely
Pooling	agregace sousedních hodnot	snižuje rozměr a zvyšuje robustnost vůči malým posunům
Receptive field	oblast vstupu, která ovlivňuje jednotku	ve vyšších vrstvách reprezentuje větší část obrazu

KONVOLUČNÍ NS

- v CNN jsou potřeba vrstvy pro zmenšení rozměrů obrázku
 - **pooling** = sdružení hodnot sousedních jednotek z předchozí vrstvy do jedné jednotky bez použití aktivační funkce --> varianty:
 - zprůměrování - výsledkem je průměrná hodnota - obrázek má hrubší rozlišení
 - max-pooling – vybere se maximální hodnota
 - pokud je cílem zařadit obrázek do jedné z **c** kategorií, pak poslední vrstva sítě bude softmax s **c** výstupními jednotkami. První vrstvy CNN mají velikost obrazu, takže někde mezi nimi musí dojít k výraznému zmenšení velikosti vrstev. Ke zmenšení velikosti vrstev slouží konvoluční vrstvy a vrstvy poolingové s krokem větším než 1. Velikost vrstvy je také možné snížit jednoduše tím, že plně propojená vrstva má méně jednotek než vrstva předchozí. CNN mají často jednu nebo dvě takové vrstvy předcházející finální vrstvě softmax.
-

ARCHITEKTURA SÍTĚ

- architektura = počet vrstev, jejich propojení, typy uzlů, aktivační funkce, typů uzlů v každé vrstvě
- návrh architektury sítě závisí na konkrétním druhu dat - obrázky, řeč, text, video, ...
- používané algoritmy:
 - evoluční
 - horolezecký, viz přednáška č. 4
- hlubší síť obvykle poskytuje lepší generalizaci a bohatší reprezentace, ale zároveň roste obtížnost trénování
- v praxi kompromis mezi přesností, rychlostí, velikostí modelu a dostupným množstvím dat
- příklad - reálná aplikace na rozpoznávání čísel domů, kdy pro pevný počet parametrů má 11-vrstvá síť pro testovací množinu nižší chybovost než 3-vrstvá



ARCHITEKTURA SÍTĚ

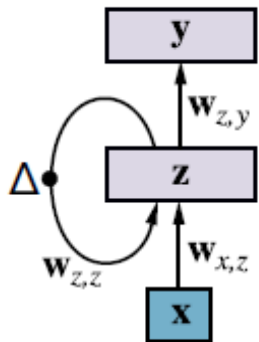
Co obvykle sledujeme při návrhu architektury

- Velikost datasetu: malý dataset zvyšuje riziko přeučení, proto je vhodná regularizace nebo transfer learning.
- Typ dat: obraz → CNN, sekvence → RNN/LSTM/transformer, tabulky → MLP nebo gradient boosting.
- Cílová úloha: klasifikace, regrese, detekce, generování – každá vyžaduje jinou ztrátu i výstupní vrstvu.
- Výpočetní omezení: na okrajových zařízeních musí být model menší a rychlejší než v serverovém prostředí.

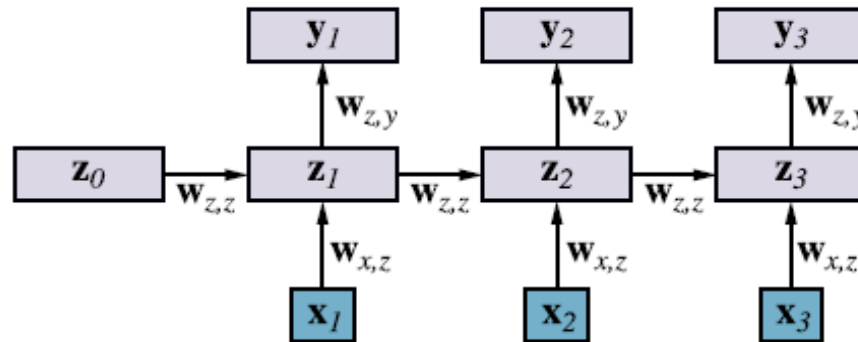
Technika	K čemu slouží	Poznámka
Dropout	náhodně vypíná část neuronů během trénování	zmenšuje spoluadaptaci a přeučení
Early stopping	zastaví trénování při zhoršení validační chyby	jednoduchá a účinná ochrana proti overfittingu
L2 regularizace	trestá příliš velké váhy	hladší model, často stabilnější učení
Batch normalization	stabilizuje rozdělení aktivací	urychluje a zpřesňuje učení hlubších sítí
Augmentace dat	uměle rozšiřuje trénovací data	zásadní zejména v počítačovém vidění

REKURENTNÍ NN (RNN)

Rekurentní neuronové sítě (RNN) se od dopředných sítí liší tím, že umožňují cykly ve výpočetním grafu. Každý cyklus má zpoždění, tak aby jako vstup do jednotky mohla být použita hodnota jejího vlastního výstupu z předchozího kroku. RNN má paměť (vnitřní stav).



(a)



(b)

(a) základní schéma RNN, kde skrytá vrstva z má rekurentní propojení

(b) stejná síť rozvinutá ve třech časových krocích, váhy jsou sdílené ve všech krocích

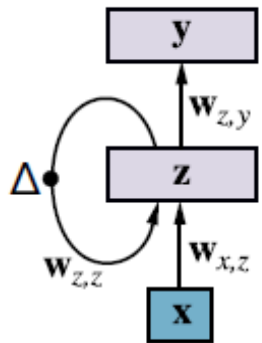
rovnice modelu

$$\begin{aligned} \mathbf{z}_t &= f_{\mathbf{w}}(\mathbf{z}_{t-1}, \mathbf{x}_t) = \mathbf{g}_z(\mathbf{W}_{z,z}\mathbf{z}_{t-1} + \mathbf{W}_{x,z}\mathbf{x}_t) \equiv \mathbf{g}_z(\mathbf{in}_{z,t}) \\ \hat{\mathbf{y}}_t &= \mathbf{g}_y(\mathbf{W}_{z,y}\mathbf{z}_t) \equiv \mathbf{g}_y(\mathbf{in}_{y,t}), \end{aligned}$$

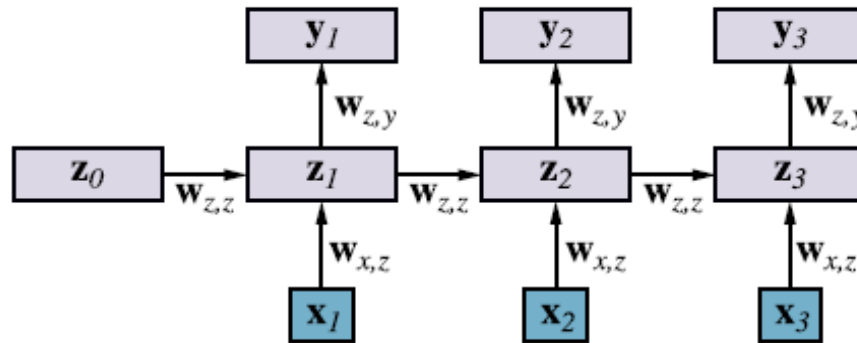
REKURENTNÍ NN (RNN)

Rekurentní neuronové sítě (RNN) zpracovávají sekvence. Na rozdíl od dopředných sítí mají vnitřní stav, který se přenáší z jednoho časového kroku do dalšího. Díky tomu může aktuální výstup záviset nejen na současném vstupu x , ale i na historii.

RNN se hodí pro text, řeč, senzory a časové řady. Síť se často zakresluje ve dvou podobách: jako jediný uzel s rekurentní smyčkou a jako rozvinutá síť přes časové kroky. V rozvinutém pohledu je dobře vidět, že se v každém kroku používají stejné váhy.



(a)



(b)

(a) základní schéma RNN, kde skrytá vrstva z má rekurentní propojení

(b) stejná síť rozvinutá ve třech časových krocích, váhy jsou sdílené ve všech krocích

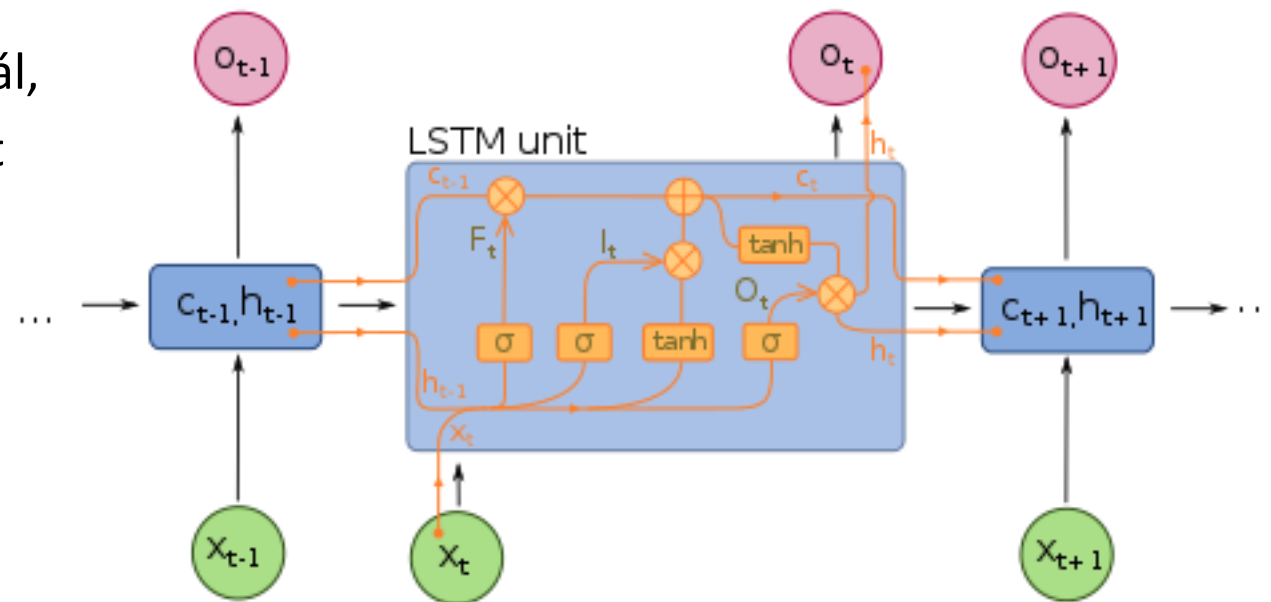
rovnice modelu

$$z_t = f_w(z_{t-1}, x_t) = g_z(W_{z,z}z_{t-1} + W_{x,z}x_t) \equiv g_z(\text{in}_{z,t})$$

$$\hat{y}_t = g_y(W_{z,y}z_t) \equiv g_y(\text{in}_{y,t}),$$

LSTM NEURONOVÉ SÍTĚ

- problém klasické RNN - při dlouhých sekvencích mohou gradienty mizet nebo explodovat a informace ze vzdálené minulosti se pak špatně přenáší - tento problém vedl ke vzniku LSTM
- LSTM (Long Short-Term Memory) je druh rekurentní NS, která zachová historii a odstraní nepodstatné informace
- LSTM přidává explicitní paměťovou buňku c_t a tři hlavní brány: zapomínací, vstupní a výstupní; každá brána rozhoduje, kolik informace projde dál, tím se síť učí co si zapamatovat a co zapomenout



LSTM NEURONOVÉ SÍTĚ

- „zapomínací“ brána f určuje, zda se prvek paměťové buňky zapamatuje (zkopíruje do dalšího časového kroku) nebo zapomene (anuluje).
 - vstupní brána i určuje, zda je v aktuálním časovém kroku prvek paměťové buňky aditivně aktualizován o nové informace ze vstupního vektoru.
 - výstupní brána o určuje, zda je prvek paměťové buňky přenesen do krátkodobé paměti z , která hraje podobnou roli jako skrytý stav v základních RNN.
 - hodnoty pro jednotky hradla jsou vždy v rozsahu $[0, 1]$ a jsou získány jako výstupy sigmoidní funkce aplikované na aktuální vstup a předchozí skrytý stav - hodnota blízká nule příslušný tok informací potlačí, hodnota blízká jedné jej ponechá téměř beze změny
-

LSTM - ROVNICE

$$\mathbf{f}_t = \sigma(\mathbf{W}_{x,f}\mathbf{x}_t + \mathbf{W}_{z,f}\mathbf{z}_{t-1})$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_{x,i}\mathbf{x}_t + \mathbf{W}_{z,i}\mathbf{z}_{t-1})$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_{x,o}\mathbf{x}_t + \mathbf{W}_{z,o}\mathbf{z}_{t-1})$$

$$\mathbf{c}_t = \mathbf{c}_{t-1} \odot \mathbf{f}_t + \mathbf{i}_t \odot \tanh(\mathbf{W}_{x,c}\mathbf{x}_t + \mathbf{W}_{z,c}\mathbf{z}_{t-1})$$

$$\mathbf{z}_t = \tanh(\mathbf{c}_t) \odot \mathbf{o}_t,$$

symbol $\odot$ značí násobení prvek po prvku

UČENÍ BEZ UČITELE

- učení bez učitele pracuje s neoznačenými daty - model nedostává správnou odpověď y , ale snaží se objevit strukturu v samotných vstupech x - to je cenné tehdy, když je označování dat drahé nebo když nás zajímá vnitřní organizace dat
 - úlohy - redukce dimenzionality, shlukování, hledání latentních faktorů, generování nových vzorů a předtrénování reprezentací pro další úlohy
 - aplikace:
 - > generativní modely --vytváření realistických textů, obrázků, zvuků a videí, nikoliv pouze předpovídání výstupů
 - > předzpracování dat pro učení s učitelem -- nalezení nových vzorů (příznaky) v obrázcích
 - příklad - obrázek na další straně - algoritmus zvolí jeden směr pro reprezentaci pohlaví a další pro reprezentaci obličeje (přítomnost nebo nepřítomnost brýlí)
-



-



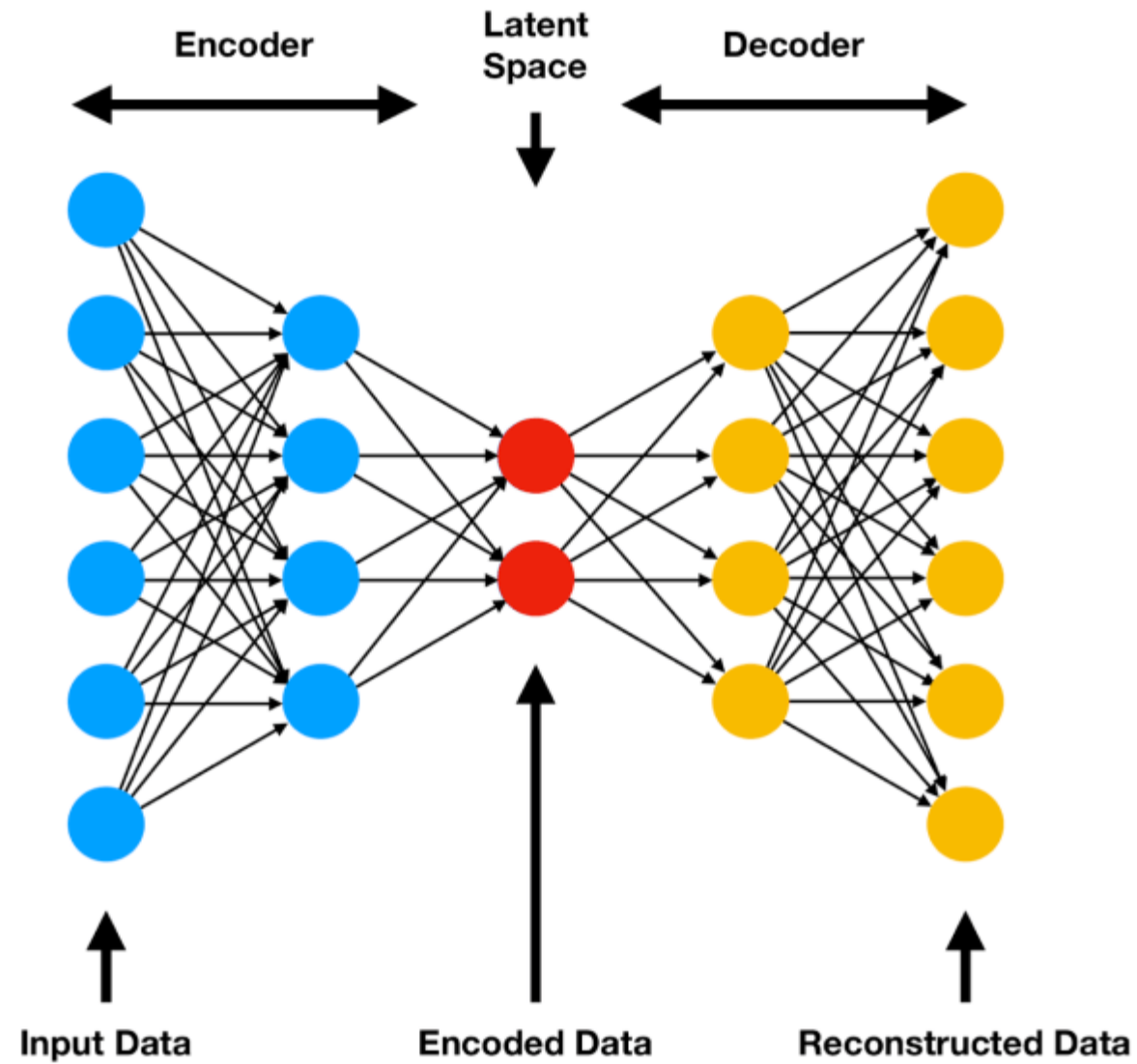
+



=



AUTOENKODÉRY



AUTOENKODÉRY

Auto-ekodér je model obsahující dvě části: kodér, který zobrazuje vstupy $\mathbf{x}$ na reprezentaci $\mathbf{z}$, a dekodér, který zobrazuje $\mathbf{z}$ zpět na pozorovaná data $\mathbf{x}$. Obecně je kodér parametrizovaná funkce $f(\mathbf{x})$ a dekodér je parametrizovaná funkce $g(\mathbf{z})$. Model je natrénován tak, aby platila přibližná rovnost $\mathbf{x} \sim g(f(\mathbf{x}))$, a tedy proces kódování je přibližně inverzní k procesu dekódování. Funkce $\mathbf{f}$ a $\mathbf{g}$ mohou být jednoduché lineární modely parametrizované jednou maticí anebo mohou být reprezentovány hlubokou neuronovou sítí.

Jednoduchý příklad: lineární auto-ekodér, kde $\mathbf{f}$ i $\mathbf{g}$ jsou lineární funkce se společnou váhovou maticí $\mathbf{W}$:

$$\begin{aligned}\hat{\mathbf{z}} &= f(\mathbf{x}) = \mathbf{W}\mathbf{x} \\ \mathbf{x} &= g(\hat{\mathbf{z}}) = \mathbf{W}^\top \hat{\mathbf{z}}.\end{aligned}$$

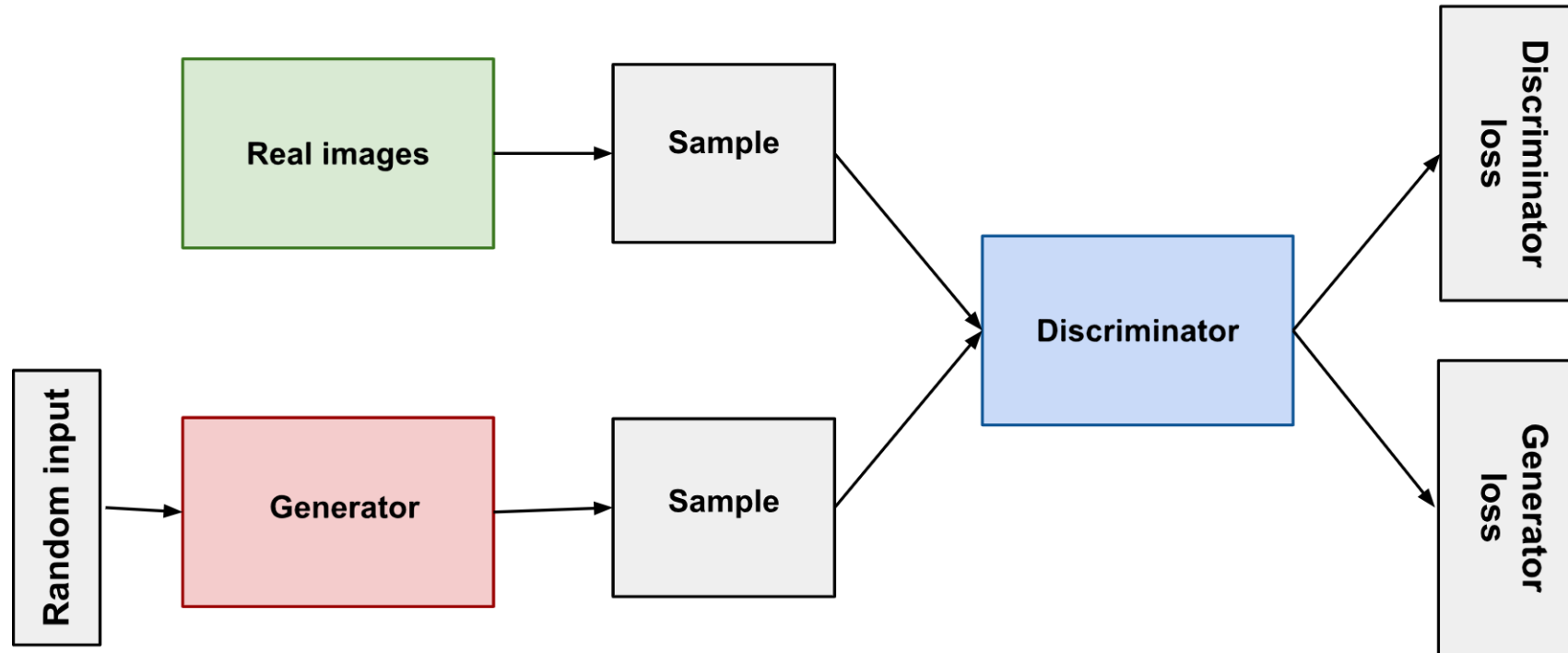
Trénování modelu spočívá v minimalizace kvadratické chyby $\sum_j \|\mathbf{x}_j - g(f(\mathbf{x}_j))\|^2$ tak, aby $\mathbf{x} \sim g(f(\mathbf{x}))$.

Cílem je, nastavit $\mathbf{W}$ tak, aby nízko-rozměrná veličina $\mathbf{z}$ zachovala co nejvíce informací a aby bylo možné zpětně rekonstruovat vysoko-rozměrná data $\mathbf{x}$.

AUTO-ENKODÉRY - APLIKACE

- komprese dat - mnoharozměrná data jsou komprimována a v případě potřeby pak zrekonstruována do původní podoby
 - „denoising“ - odstranění šumu z dat, obnovení původních dat z poškozených záznamů
 - extrakce příznaků z dat
 - generativní modelování - variační auto-ekodéry (VAE), vytváření nových datových vzorů → tvorba obsahu nebo uměleckého generování
 - snížení dimenzionality - pro vizualizaci a zrychlení trénování modelů
 - před-trénování modelu nebo části modelu a následné využití pro úlohy klasifikace; zlepšuje výkonnost modelů v případech s omezeným množstvím trénovacích dat
-

GAN - GENERATIVE ADVERSARIAL NETWORK



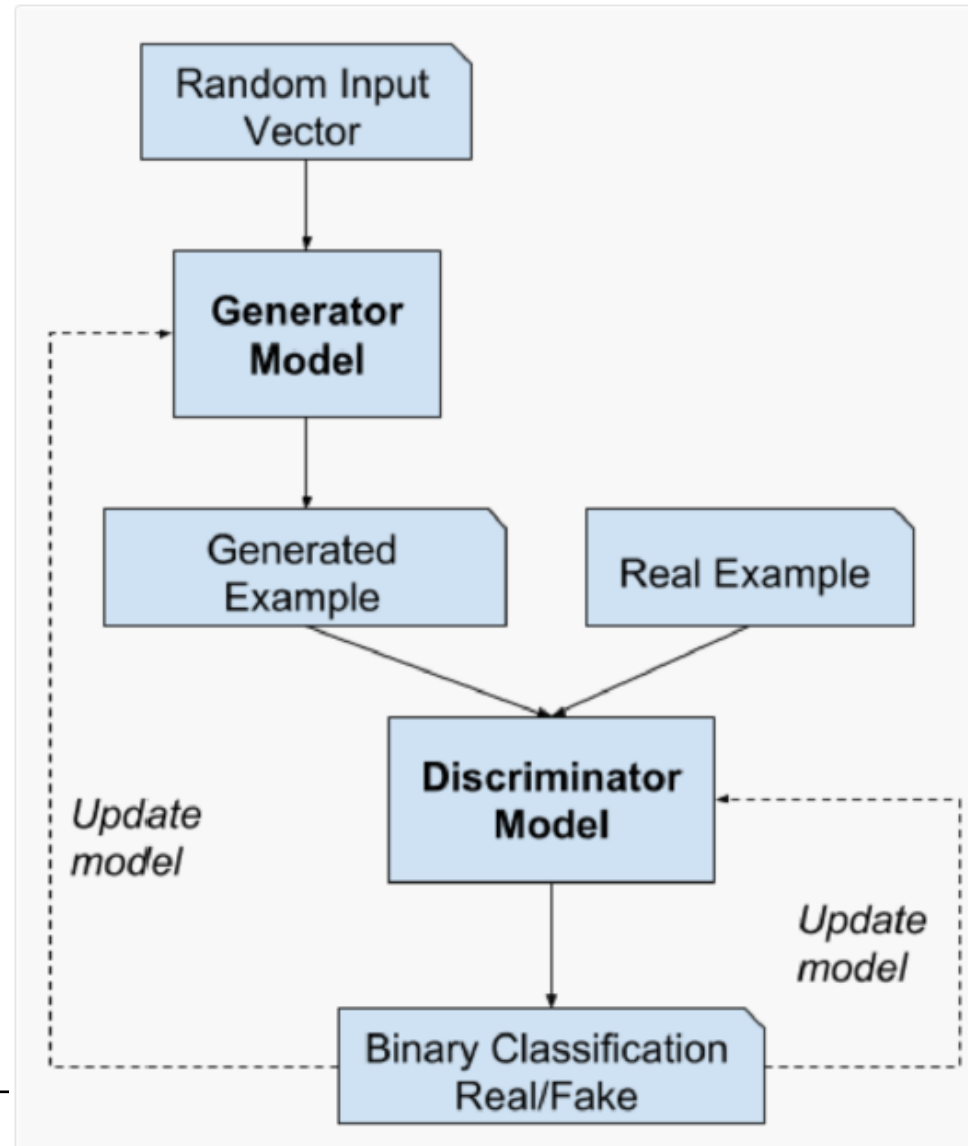
GAN - GENERATIVE ADVERSARIAL NETWORK

- dvojice sítí:
 - **generátor** vytváří nová data
 - **diskriminátor** rozhoduje, zda jsou data vytvořená generátorem pravdivá (tj. zda jsou stejná jako data ve vstupním datasetu) nebo falešná (tj. vytvořená generátorem).
 - Proces trénování GAN sítě:
 1. Generátor dostává na vstup náhodný šum a na výstup generuje nová data.
 2. Diskriminátor dostává na vstup buď data z vstupního datasetu nebo data generovaná generátorem.
 3. Diskriminátor rozhodne, zda jsou data pravdivá nebo falešná.
 4. Chyba výstupu diskriminátoru se používá k aktualizaci parametrů diskriminátoru, aby byl schopen lépe rozlišovat mezi pravdivými a falešnými daty.
 5. Chyba generovaných dat se používá k aktualizaci parametrů generátoru, aby byl schopen generovat lepší data, která budou přijata diskriminátorem.

Tento proces se opakuje, dokud není dosaženo dostatečné úrovně kvality generovaných dat.

Poté lze generátor použít k vytváření nových dat, která jsou velmi podobná datům v trénovacím data-setu.
-

GAN - GENERATIVE ADVERSARIAL NETWORK



GAN - APLIKACE

- generativní modelování - vytváření realistických dat - obrázky, fotografie, umělecká díla, ...
 - vylepšování obrázků - zlepšení kvality obrázků, např. převod obrázků nízké kvality na obrázky vyšší kvality
 - stylizace obrázků - aplikace různých uměleckých stylů a efektů na originální obrázky
 - generování realistických obličejů pro herní průmysl a film
 - generování řeči nebo napodobení lidského hlasu (syntéza řeči)
 - augmentace dat - generování nových trénovacích dat (pro úlohy učení s učitelem), pokud jsou originální data těžko dostupná
 - vytváření textur a vzorů v oblasti grafického designu
-

Zdroje:

Russell, S. J., Norvig, P. *Artificial Intelligence: A Modern Approach*. 4. vyd. Pearson, 2021

<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

<https://machinelearningmastery.com/what-are-generative-adversarial-networks-gans/>

<https://www.geeksforgeeks.org/generative-adversarial-network-gan/>

https://developers.google.com/machine-learning/gan/gan_structure
